EX NO : 6 Date : 24/09/26	FEEDFORWARD NEURAL NETWORK AND CONVOLUTIONAL NEURAL NETWORK

AIM:

To implement and evaluate **Feedforward Neural Network (FNN)** and **Convolutional Neural Network (CNN)** models using **TensorFlow/Keras** for image classification using the **Fashion-MNIST and MNIST datasets**.

ALGORITHM

Part 1: Feedforward Neural Network (FNN)

1. Start
2. Import the required libraries such as NumPy, TensorFlow, Keras, Matplotlib, and Scikit-learn.
3. Load the Fashion-MNIST dataset.
4. Split the dataset into training and testing data.
5. Flatten each 28 × 28 image into a 784-element vector.
6. Normalize the pixel values by dividing them by 255.
7. Create an FNN model with:
    - Dense layer with 128 neurons and ReLU activation.
    - Dropout layer. o Dense layer with 64 neurons and ReLU activation.
    - Output Dense layer with 10 neurons and Softmax activation.
8. Compile the model using the Adam optimizer and sparse categorical cross-entropy loss.
9. Train the model for 10 epochs.
10. Evaluate the trained model using test data.
11. Generate predictions and calculate the classification report and confusion matrix.
12. Plot training and validation accuracy and loss.
13. End FNN.

Part 2: Convolutional Neural Network (CNN)

1. Start
2. Load the MNIST dataset.
3. Split the dataset into training and testing data.
4. Reshape the images into 28 × 28 × 1 format.
5. Normalize the pixel values by dividing them by 255.
6. Create a CNN model with:
    - Conv2D layer with 32 filters.
    - MaxPooling2D layer.
    - Conv2D layer with 64 filters. o MaxPooling2D layer.
    - Flatten layer.
    - Dense layer with 128 neurons and ReLU activation. o Dropout layer.
    - Output Dense layer with 10 neurons and Softmax activation.
7. Compile the model using the Adam optimizer and sparse categorical cross-entropy loss.
8. Train the CNN model for 10 epochs.
9. Evaluate the model using test data.
10. Generate predictions and calculate the classification report and confusion matrix.
11. Plot training and validation accuracy and loss.
12. Display sample images with their actual and predicted labels.
13. End CNN.

CODE:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import mnist, fashion_mnist
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns

tf.keras.utils.disable_interactive_logging()



print("--- Part 1: Building a Simple Feedforward Neural Network ---")

(x_train_fnn, y_train_fnn), (x_test_fnn, y_test_fnn) = fashion_mnist.load_data()

print("\nOriginal FNN training data shape:", x_train_fnn.shape)
print("Original FNN test data shape:", x_test_fnn.shape)


x_train_fnn_flat = x_train_fnn.reshape(-1, 28 * 28)
x_test_fnn_flat = x_test_fnn.reshape(-1, 28 * 28)


x_train_fnn_norm = x_train_fnn_flat / 255.0
x_test_fnn_norm = x_test_fnn_flat / 255.0

print("Flattened & Normalized FNN training data shape:",
      x_train_fnn_norm.shape)

print("Flattened & Normalized FNN test data shape:",
      x_test_fnn_norm.shape)


model_fnn = keras.Sequential([
    layers.Dense(128, activation='relu', input_shape=(784,)),
    layers.Dropout(0.2),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

model_fnn.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
```

```
print("\n--- Training FNN Model ---")

history_fnn = model_fnn.fit(
    x_train_fnn_norm,
    y_train_fnn,
    epochs=10,
    validation_split=0.1,
    verbose=1
)


print("\n--- Evaluating FNN Model ---")

loss_fnn, accuracy_fnn = model_fnn.evaluate(
    x_test_fnn_norm,
    y_test_fnn,
    verbose=0
)

print(f"FNN Test Loss: {loss_fnn:.4f}")
print(f"FNN Test Accuracy: {accuracy_fnn:.4f}")


y_pred_fnn = np.argmax(
    model_fnn.predict(x_test_fnn_norm, verbose=0),
    axis=-1
)


print("\n--- FNN Classification Report ---")
print(classification_report(y_test_fnn, y_pred_fnn))


print("\n--- FNN Confusion Matrix ---")

cm_fnn = confusion_matrix(y_test_fnn, y_pred_fnn)

plt.figure(figsize=(10, 8))
sns.heatmap(
    cm_fnn,
    annot=True,
    fmt="d",
    cmap="Blues",
    cbar=False
```

```
plt.title("FNN Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()


plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)

plt.plot(
    history_fnn.history['accuracy'],
    label='Training Accuracy'
)

plt.plot(
    history_fnn.history['val_accuracy'],
    label='Validation Accuracy'
)

plt.title('FNN Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)


plt.subplot(1, 2, 2)

plt.plot(
    history_fnn.history['loss'],
    label='Training Loss'
)

plt.plot(
    history_fnn.history['val_loss'],
    label='Validation Loss'
)

plt.title('FNN Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
```

```
print("\n--- Part 2: Implementing a CNN ---")


(x_train_cnn, y_train_cnn), (x_test_cnn, y_test_cnn) = mnist.load_data()

print("\nOriginal CNN training data shape:", x_train_cnn.shape)
print("Original CNN test data shape:", x_test_cnn.shape)


x_train_cnn = x_train_cnn.reshape(
    x_train_cnn.shape[0], 28, 28, 1
)

x_test_cnn = x_test_cnn.reshape(
    x_test_cnn.shape[0], 28, 28, 1
)


x_train_cnn = x_train_cnn.astype('float32') / 255.0
x_test_cnn = x_test_cnn.astype('float32') / 255.0

print(
    "Reshaped & Normalized CNN training data shape:",
    x_train_cnn.shape
)

print(
    "Reshaped & Normalized CNN test data shape:",
    x_test_cnn.shape
)


num_classes_cnn = 10



model_cnn = keras.Sequential([

    layers.Conv2D(
        32,
        (3, 3),
        activation='relu',
        input_shape=(28, 28, 1)
    ),
```

```
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(
        64,
        (3, 3),
        activation='relu'
    ),

    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),

    layers.Dense(
        128,
        activation='relu'
    ),

    layers.Dropout(0.5),

    layers.Dense(
        num_classes_cnn,
        activation='softmax'
    )
])


model_cnn.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

print("\n--- CNN Model Summary ---")
model_cnn.summary()


print("\n--- Training CNN Model ---")

history_cnn = model_cnn.fit(
    x_train_cnn,
    y_train_cnn,
    epochs=10,
    validation_split=0.1,
    verbose=1
)
```

```
print("\n--- Evaluating CNN Model ---")

loss_cnn, accuracy_cnn = model_cnn.evaluate(
    x_test_cnn,
    y_test_cnn,
    verbose=0
)

print(f"CNN Test Loss: {loss_cnn:.4f}")
print(f"CNN Test Accuracy: {accuracy_cnn:.4f}")


y_pred_cnn = np.argmax(
    model_cnn.predict(x_test_cnn, verbose=0),
    axis=-1
)

print("\n--- CNN Classification Report ---")
print(classification_report(y_test_cnn, y_pred_cnn))


print("\n--- CNN Confusion Matrix ---")

cm_cnn = confusion_matrix(
    y_test_cnn,
    y_pred_cnn
)

plt.figure(figsize=(10, 8))

sns.heatmap(
    cm_cnn,
    annot=True,
    fmt="d",
    cmap="Blues",
    cbar=False
)

plt.title("CNN Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()


plt.figure(figsize=(12, 5))
```

```
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()


plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)

plt.plot(
    history_cnn.history['accuracy'],
    label='Training Accuracy'
)

plt.plot(
    history_cnn.history['val_accuracy'],
    label='Validation Accuracy'
)

plt.title('CNN Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)


plt.subplot(1, 2, 2)

plt.plot(
    history_cnn.history['loss'],
    label='Training Loss'
)

plt.plot(
    history_cnn.history['val_loss'],
    label='Validation Loss'
)

plt.title('CNN Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```

```
print("\n--- Sample CNN Predictions ---")

class_names_mnist = [str(i) for i in range(10)]

plt.figure(figsize=(10, 10))

for i in range(25):

    plt.subplot(5, 5, i + 1)

    plt.xticks([])
    plt.yticks([])
    plt.grid(False)

    plt.imshow(
        x_test_cnn[i].reshape(28, 28),
        cmap=plt.cm.binary
    )

    true_label = y_test_cnn[i]
    predicted_label = y_pred_cnn[i]


    color = (
        'green'
        if true_label == predicted_label
        else 'red'
    )

    plt.xlabel(
        f"True: {class_names_mnist[true_label]}\n"
        f"Pred: {class_names_mnist[predicted_label]}",
        color=color
    )


plt.suptitle(
    "Sample CNN Predictions "
    "(Green: Correct, Red: Incorrect)",
    y=1.02,
    fontsize=16
)

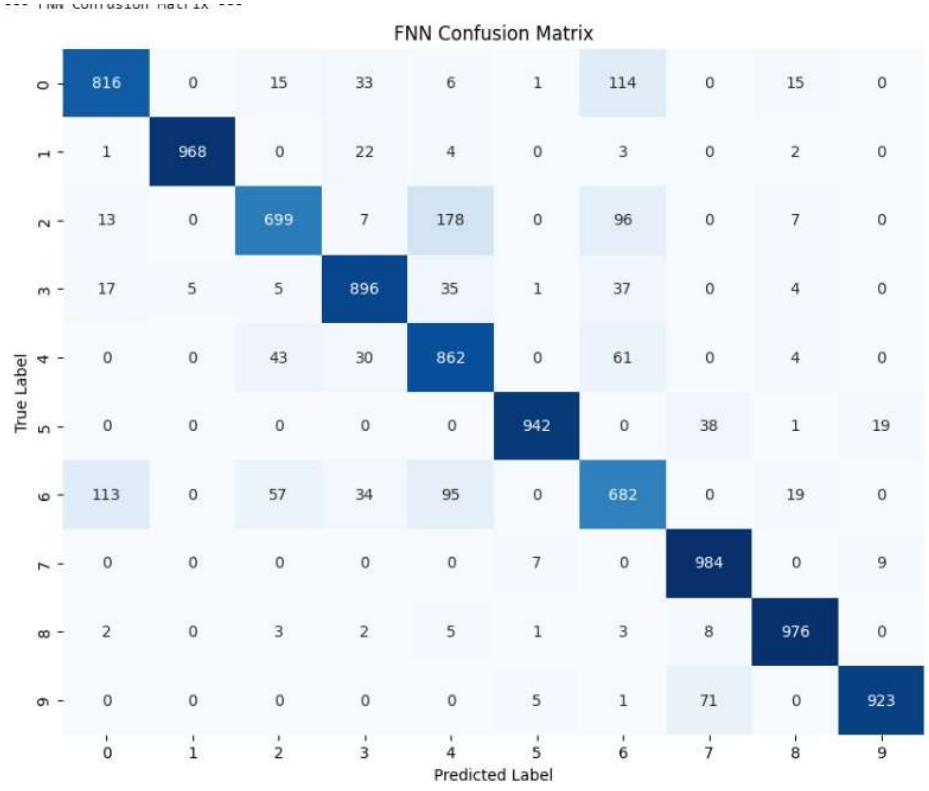
plt.tight_layout(
    rect=[0, 0, 1, 0.98]
```

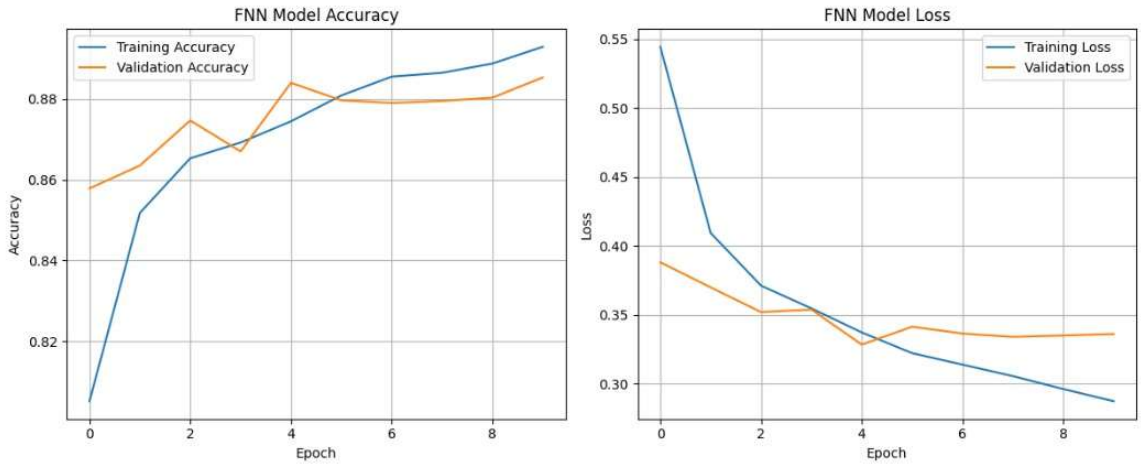
OUTPUT:

```
--- FNN Classification Report ---
              precision    recall  f1-score   support

           0       0.85      0.82      0.83      1000
           1       0.99      0.97      0.98      1000
           2       0.85      0.70      0.77      1000
           3       0.88      0.90      0.89      1000
           4       0.73      0.86      0.79      1000
           5       0.98      0.94      0.96      1000
           6       0.68      0.68      0.68      1000
           7       0.89      0.98      0.94      1000
           8       0.95      0.98      0.96      1000
           9       0.97      0.92      0.95      1000

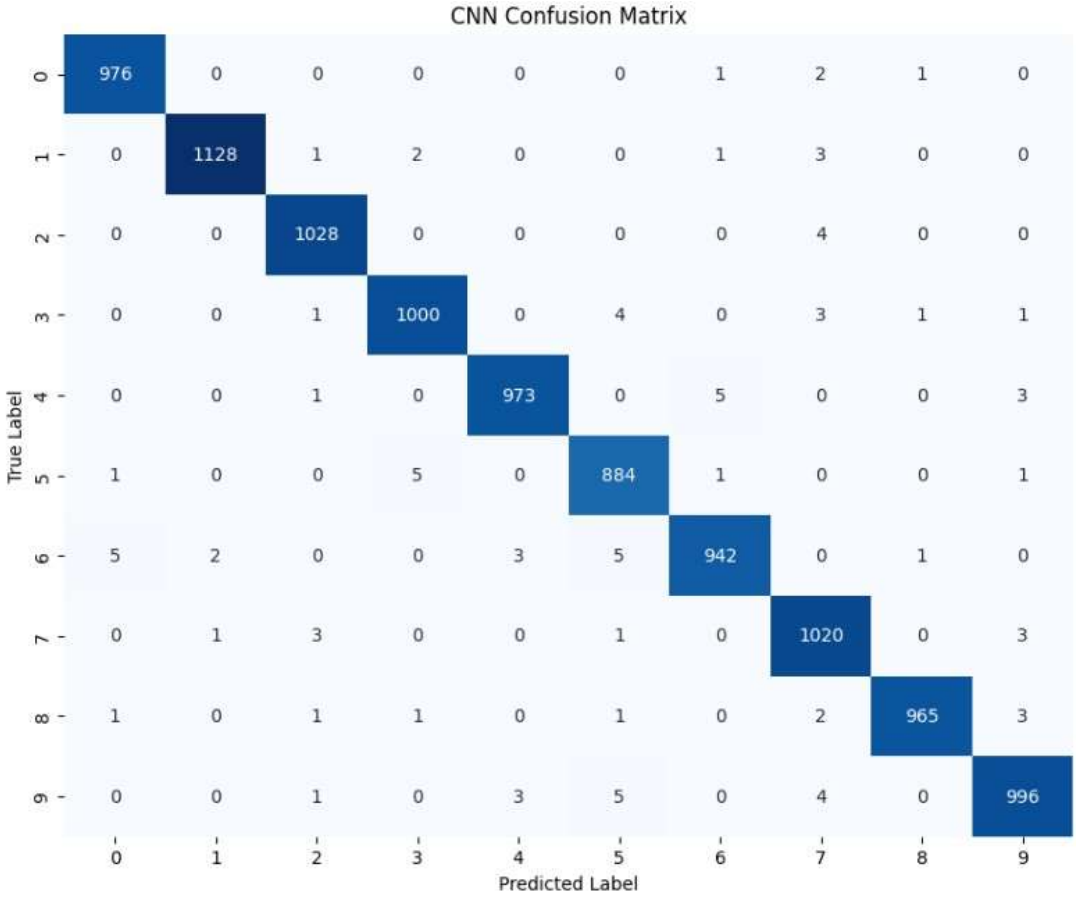
    accuracy                           0.87     10000
   macro avg       0.88      0.87      0.87     10000
weighted avg       0.88      0.87      0.87     10000
```

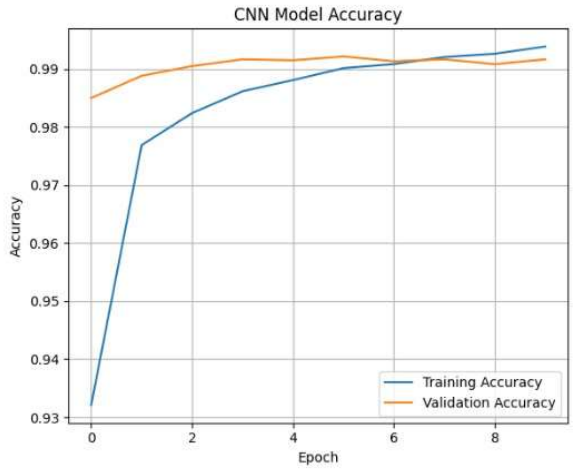


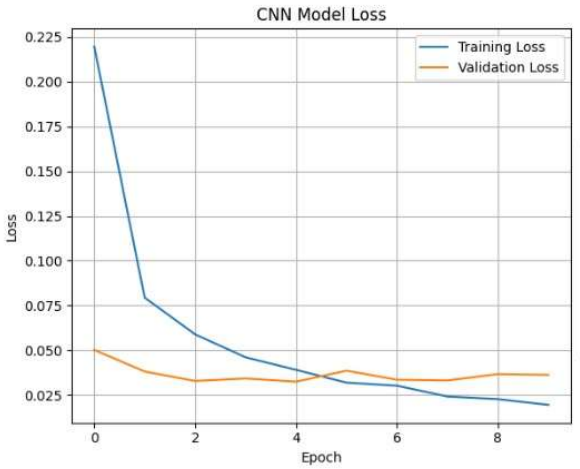


--- CNN Classification Report ---

	precision	recall	f1-score	support
0	0.99	1.00	0.99	980
1	1.00	0.99	1.00	1135
2	0.99	1.00	0.99	1032
3	0.99	0.99	0.99	1010
4	0.99	0.99	0.99	982
5	0.98	0.99	0.99	892
6	0.99	0.98	0.99	958
7	0.98	0.99	0.99	1028
8	1.00	0.99	0.99	974
9	0.99	0.99	0.99	1009
accuracy			0.99	10000
macro avg	0.99	0.99	0.99	10000
weighted avg	0.99	0.99	0.99	10000



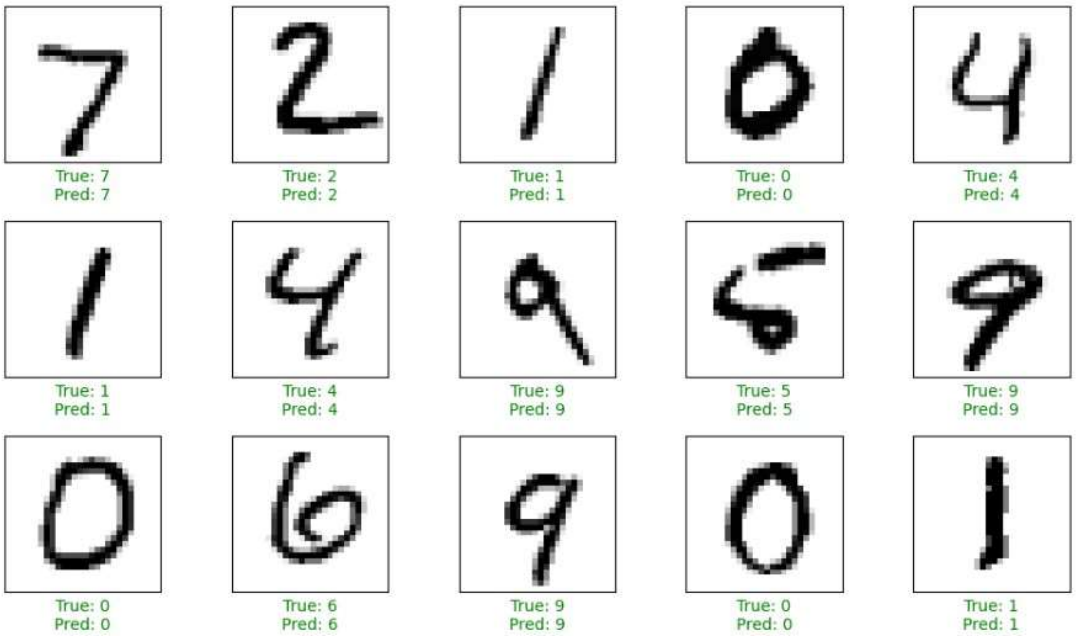


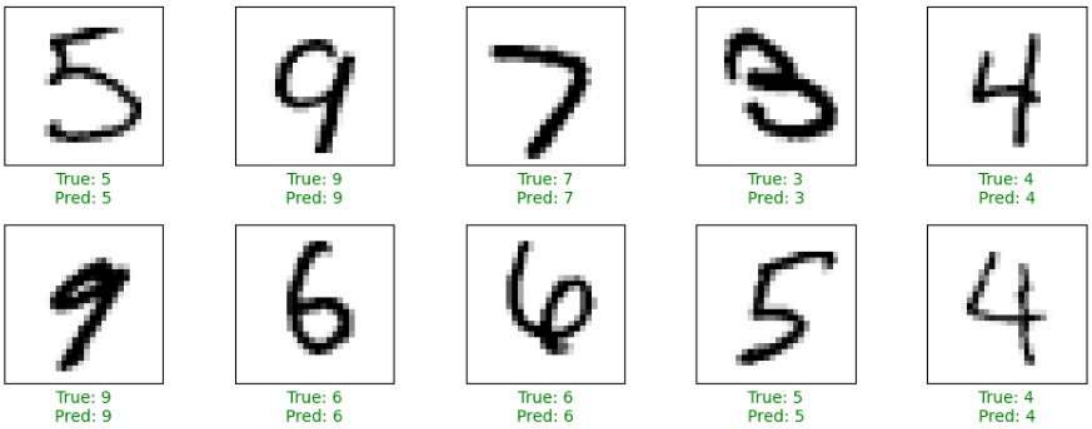



```
--- Sample CNN Predictions ---
```

Sample CNN Predictions (Green: Correct, Red: Incorrect)

Sample CNN Predictions (Green: Correct, Red: Incorrect)





RESULT:

The **FEEDFORWARD AND CONVOLUTIONAL NEURAL NETWORKS** techniques were

successfully implemented.